

Отчёт по лабораторной работе №6

Управление процессами

Ришард Когенгар

Содержание

1	Цель работы	5
2	Ход выполнения	6
2.1	Управление заданиями	6
2.2	Управление процессами	9
2.3	Задание 1. Работа с приоритетами процессов	11
2.4	Задание 2. Управление процессами с программой yes	12
3	Контрольные вопросы	17
4	Заключение	19

Список иллюстраций

2.1	Управление заданиями	7
2.2	Мониторинг процесса dd	8
2.3	Завершение процесса dd	9
2.4	Отображение процессов dd	10
2.5	Отображение иерархии процессов	11
2.6	ЗРабота с приоритетами dd	12
2.7	yес без перенаправления	13
2.8	Управление заданиями	13
2.9	Мониторинг процессов top	14
2.10	Завершение процессов	15
2.11	Выравнивание приоритетов	16

Список таблиц

1 Цель работы

Получить навыки управления процессами операционной системы.

2 Ход выполнения

2.1 Управление заданиями

1. Получены права администратора с помощью команды **su**.

После ввода пароля открыт доступ к выполнению команд от имени супер-пользователя.

2. Запущены процессы в фоновом режиме:

- `sleep 3600 &` (ожидание 1 час);
- `dd if=/dev/zero of=/dev/null &` (копирование пустого потока в «чёрную дыру»).

Дополнительно выполнена команда `sleep 7200` без символа `&`, поэтому оболочка заблокировалась до её завершения.

3. Процесс ожидания **sleep 7200** был прерван сочетанием клавиш **Ctrl+Z**, в результате чего он перешёл в состояние **Stopped**.

4. С помощью команды **jobs** отображён список фоновых заданий.

Первые два процесса находятся в состоянии **Running**, а третий был приостановлен.

5. При помощи команды **bg 4** задание продолжено в фоновом режиме.

Повторная проверка через **jobs** показала, что все три процесса выполняются.

6. Выполнена команда **fg 1**, чтобы вернуть первый процесс на передний план.
С помощью **Ctrl+C** задание было завершено.
7. Аналогичным образом отменены оставшиеся задания (**fg 2**, **fg 3**).
Работа каждого процесса завершена.

```
rkogengar@rishardkogengar:~$ su
Password:
root@rishardkogengar:/home/rkogengar# sleep 3600 &
[1] 3447
root@rishardkogengar:/home/rkogengar# dd if=/dev/zerp of=/dev/null &
[2] 3500
dd: failed to open '/dev/zerp': No such file or directory
root@rishardkogengar:/home/rkogengar# dd if=/dev/zero of=/dev/null &
[3] 3514
[2] Exit 1 dd if=/dev/zerp of=/dev/null
root@rishardkogengar:/home/rkogengar# sleep 7200
^Z
[4]+ Stopped sleep 7200
root@rishardkogengar:/home/rkogengar# jobs
[1] Running sleep 3600 &
[3]- Running dd if=/dev/zero of=/dev/null &
[4]+ Stopped sleep 7200
root@rishardkogengar:/home/rkogengar# bg 4
[4]+ sleep 7200 &
root@rishardkogengar:/home/rkogengar# jobs
[1] Running sleep 3600 &
[3]- Running dd if=/dev/zero of=/dev/null &
[4]+ Running sleep 7200 &
root@rishardkogengar:/home/rkogengar# fg 1
sleep 3600
^C
root@rishardkogengar:/home/rkogengar# fg 2
bash: fg: 2: no such job
root@rishardkogengar:/home/rkogengar# fg 3
dd if=/dev/zero of=/dev/null
^C86268489+0 records in
86268488+0 records out
44169465856 bytes (44 GB, 41 GiB) copied, 58.8239 s, 751 MB/s
root@rishardkogengar:/home/rkogengar# fg 4
sleep 7200
^C
```

Рис. 2.1: Управление заданиями

8. В новом терминале, под пользователем **rkogengar**, был запущен процесс **dd if=/dev/zero of=/dev/null &**.
После выхода из терминала (**exit**) процесс остался активным.
9. В другом терминале была выполнена команда **top**, что позволило отследить активность процессов.

В списке видно, что процесс **dd** продолжает работу и активно потребляет ресурсы CPU.

```
rkogengar@rishardkogengar:~ -- top
rkogengar@rishardkogengar:/home/rkogengar
rkogengar@rishardkogengar:~ -- top

top - 18:02:20 up 6 min, 4 users, load average: 0.24, 0.26, 0.13
Tasks: 260 total, 2 running, 258 sleeping, 0 stopped, 0 zombie
%Cpu(s): 5.4 us, 6.8 sy, 0.0 ni, 87.4 id, 0.0 wa, 0.3 hi, 0.1 si, 0.0 st
MiB Mem : 3909.0 total, 1304.7 free, 1425.3 used, 1418.4 buff/cache
MiB Swap: 4040.0 total, 4040.0 free, 0.0 used, 2483.7 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR S %CPU  %MEM    TIME+  COMMAND
 3928 rkogeng+  20   0 226848 1776 1776 R 99.7   0.0   0:09.91 dd
 2108 rkogeng+  20   0 4908712 309436 122288 S 2.3   7.7   0:03.74 gnome-shell
 3224 rkogeng+  20   0 3029364 303952 96260 S 2.0   7.6   0:02.81 ptxixis
  123 root       20   0      0      0      0 I 0.3   0.0   0:00.11 kworker/u18:2-events_unbo+
    1 root       20   0 49192 41172 10248 S 0.0   1.0   0:01.47 systemd
    2 root       20   0      0      0      0 S 0.0   0.0   0:00.00 kthreadd
    3 root       20   0      0      0      0 S 0.0   0.0   0:00.00 pool_workqueue_release
    4 root       0 -20      0      0      0 I 0.0   0.0   0:00.00 kworker/R-rcu_gp
    5 root       0 -20      0      0      0 I 0.0   0.0   0:00.00 kworker/R-sync_wq
    6 root       0 -20      0      0      0 I 0.0   0.0   0:00.00 kworker/R-slub_flushwq
    7 root       0 -20      0      0      0 I 0.0   0.0   0:00.00 kworker/R-netns
    9 root       20   0      0      0      0 I 0.0   0.0   0:00.07 kworker/0:1-events
   10 root       0 -20      0      0      0 I 0.0   0.0   0:00.00 kworker/0:0H-events_highp+
   11 root       20   0      0      0      0 I 0.0   0.0   0:00.00 kworker/u16:0-events_unbo+
   12 root       20   0      0      0      0 I 0.0   0.0   0:00.02 kworker/u16:1-netns
   13 root       0 -20      0      0      0 I 0.0   0.0   0:00.00 kworker/R-mm_percpu_wq
   14 root       20   0      0      0      0 I 0.0   0.0   0:00.00 rcu_tasks_kthread
   15 root       20   0      0      0      0 I 0.0   0.0   0:00.00 rcu_tasks_rude_kthread
   16 root       20   0      0      0      0 I 0.0   0.0   0:00.00 rcu_tasks_trace_kthread
   17 root       20   0      0      0      0 S 0.0   0.0   0:00.00 ksoftirqd/0
   18 root       20   0      0      0      0 I 0.0   0.0   0:00.06 rcu_preempt
   19 root       20   0      0      0      0 S 0.0   0.0   0:00.00 rcu_exp_par_gp_kthread_wo+
   20 root       20   0      0      0      0 S 0.0   0.0   0:00.08 rcu_exp_gp_kthread_worker
   21 root       rt   0      0      0      0 S 0.0   0.0   0:00.01 migration/0
   22 root      -51   0      0      0      0 S 0.0   0.0   0:00.00 idle_inject/0
   23 root       20   0      0      0      0 S 0.0   0.0   0:00.00 cpuhp/0
   24 root       20   0      0      0      0 S 0.0   0.0   0:00.00 cpuhp/1
```

Рис. 2.2: Мониторинг процесса dd

10. Снова открыта утилита **top**, и с помощью клавиши **k** процесс **dd** был завершён.

После этого отображение обновилось, и задание исчезло из списка.

```
rkogengar@rishardkogengar:~ - top
rkogengar@rishardkogengar:/home/rkogengar
rkogengar@rishardkogengar:~ - top

top - 18:02:41 up 6 min, 4 users, load average: 0.66, 0.36, 0.16
Tasks: 260 total, 1 running, 259 sleeping, 0 stopped, 0 zombie
%Cpu(s): 5.0 us, 6.5 sy, 0.1 ni, 88.3 id, 0.0 wa, 0.2 hi, 0.0 si, 0.0 st
MiB Mem : 3909.0 total, 1290.4 free, 1439.3 used, 1418.8 buff/cache
MiB Swap: 4040.0 total, 4040.0 free, 0.0 used, 2469.7 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
3224	rkogeng+	20	0	3029364	323536	96260	S	2.4	8.1	0:03.09	ptxis
2108	rkogeng+	20	0	4912024	309820	122288	S	0.6	7.7	0:03.93	gnome-shell
1	root	20	0	49192	41172	10248	S	0.3	1.0	0:01.50	systemd
195	root	20	0	0	0	0	I	0.3	0.0	0:00.08	kworker/u18:3-events_unbo+
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthreadd
3	root	20	0	0	0	0	S	0.0	0.0	0:00.00	pool_workqueue_release
4	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-rcu_gp
5	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-sync_wq
6	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-slab_flushwq
7	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-netns
9	root	20	0	0	0	0	I	0.0	0.0	0:00.07	kworker/0:1-events
10	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/0:0H-events_highp+
11	root	20	0	0	0	0	I	0.0	0.0	0:00.00	kworker/u16:0-events_unbo+
12	root	20	0	0	0	0	I	0.0	0.0	0:00.03	kworker/u16:1-netns
13	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-mm_percpu_wq
14	root	20	0	0	0	0	I	0.0	0.0	0:00.00	rcu_tasks_kthread
15	root	20	0	0	0	0	I	0.0	0.0	0:00.00	rcu_tasks_rude_kthread
16	root	20	0	0	0	0	I	0.0	0.0	0:00.00	rcu_tasks_trace_kthread
17	root	20	0	0	0	0	S	0.0	0.0	0:00.00	ksoftirqd/0
18	root	20	0	0	0	0	I	0.0	0.0	0:00.07	rcu_preempt
19	root	20	0	0	0	0	S	0.0	0.0	0:00.00	rcu_exp_par_gp_kthread_wo+
20	root	20	0	0	0	0	S	0.0	0.0	0:00.08	rcu_exp_gp_kthread_worker
21	root	rt	0	0	0	0	S	0.0	0.0	0:00.01	migration/0
22	root	-51	0	0	0	0	S	0.0	0.0	0:00.00	idle_inject/0
23	root	20	0	0	0	0	S	0.0	0.0	0:00.00	cpuhp/0
24	root	20	0	0	0	0	S	0.0	0.0	0:00.00	cpuhp/1
25	root	-51	0	0	0	0	S	0.0	0.0	0:00.00	idle_inject/1

Рис. 2.3: Завершение процесса dd

2.2 Управление процессами

1. Получены права администратора с помощью команды **su**.

Это позволило выполнять системные операции с повышенными привилегиями.

2. Запущено три ресурсоёмких процесса командой **dd if=/dev/zero of=/dev/null &**.

Каждый процесс выполнялся в фоне, записывая данные в «чёрную дыру» (**/dev/null**).

3. Для просмотра списка активных процессов была выполнена команда **ps aux | grep dd**.

В выводе отобразились процессы, содержащие в команде подстроку *dd*.

Запущенные процессы оказались в конце списка.

```
root@rishaardkogengar:/home/rkogengar#  
root@rishaardkogengar:/home/rkogengar# dd if=/dev/zero of=/dev/null &  
[1] 4485  
root@rishaardkogengar:/home/rkogengar# dd if=/dev/zero of=/dev/null &  
[2] 4497  
root@rishaardkogengar:/home/rkogengar# dd if=/dev/zero of=/dev/null &  
[3] 4499  
root@rishaardkogengar:/home/rkogengar# ps aux | grep dd  
root      2  0.0  0.0   0   0 ?        S   17:56   0:00 [kthreadd]  
root     91  0.0  0.0   0   0 ?        I<  17:56   0:00 [kworker/R-ipv6_addrconf]  
root    1151  0.0  0.0 512956 3012 ?        Sl   17:56   0:00 /usr/sbin/VBoxService --pidfile  
/var/run/vboxadd-service.sh  
rkogeng+ 2510  0.0  0.6 962680 25188 ?        Ssl  17:56   0:00 /usr/libexec/evolution-addressb  
ook-factory  
root     4485 99.4  0.0 226848 1900 pts/0    R   18:06   0:17 dd if=/dev/zero of=/dev/null  
root     4497 99.2  0.0 226848 1756 pts/0    R   18:06   0:12 dd if=/dev/zero of=/dev/null  
root     4499 98.5  0.0 226848 1872 pts/0    R   18:06   0:10 dd if=/dev/zero of=/dev/null  
root     4535  0.0  0.0 227688 2176 pts/0    S+  18:06   0:00 grep --color=auto dd  
root@rishaardkogengar:/home/rkogengar#
```

Рис. 2.4: Отображение процессов dd

- Один из процессов был выбран для изменения приоритета с помощью команды **renice**.

Приоритет был понижен на 5 единиц, что изменило его положение в планировщике задач.

- Для анализа иерархии процессов использована команда **ps fax | grep -B5 dd**.

Опция -B5 показала процессы вместе с пятью строками выше совпадения. Благодаря этому отобразилась цепочка: оболочка → su → bash → процессы dd.

```

    958 ?      Ssl  0:00 /usr/bin/python3 -sP /usr/sbin/firewalld --nofork --nopid
    1149 ?     Sl   0:00 /usr/bin/VBoxDRMClient
    1151 ?     Sl   0:00 /usr/sbin/VBoxService --pidfile /var/run/vboxadd-service.sh

--
    2453 ?     Ssl  0:00 \_ /usr/libexec/goa-identity-service
    2456 ?     Ssl  0:00 \_ /usr/libexec/evolution-calendar-factory
    2469 ?     Ssl  0:00 \_ /usr/libexec/gvfs-mtp-volume-monitor
    2483 ?     Ssl  0:00 \_ /usr/libexec/gvfs-gphoto2-volume-monitor
    2505 ?     Ssl  0:00 \_ /usr/libexec/gvfs-goa-volume-monitor
    2510 ?     Ssl  0:00 \_ /usr/libexec/evolution-addressbook-factory

--
    3224 ?     Ssl  0:05 \_ /usr/bin/ptxixis --gaplication-service
    3232 ?     Ssl  0:00 | \_ /usr/libexec/ptxixis-agent --socket-fd=3
    3297 pts/0  Ss   0:00 | \_ /usr/bin/bash
    3359 pts/0  S    0:00 | \_ su
    3394 pts/0  S    0:00 | \_ bash
    4485 pts/0  RN   0:48 | \_ dd if=/dev/zero of=/dev/null
    4497 pts/0  R    0:43 | \_ dd if=/dev/zero of=/dev/null
    4499 pts/0  R    0:42 | \_ dd if=/dev/zero of=/dev/null
    4603 pts/0  R+   0:00 | \_ ps fax
    4604 pts/0  S+   0:00 | \_ grep --color=auto -B5 dd

root@rishardkogengar:/home/rkogengar#

```

Рис. 2.5: Отображение иерархии процессов

- По найденному PID корневой оболочки выполнена команда **kill -9**.

В результате оболочка завершила работу, а вместе с ней были остановлены и все дочерние процессы dd.

Такой подход позволяет завершить сразу целую группу связанных процессов.

2.3 Задание 1. Работа с приоритетами процессов

- Запущено три фоновых процесса с помощью команды `dd if=/dev/zero of=/dev/null &`.

Каждый из них выполнялся независимо, загружая процессор.

- С помощью команды **renice -n 5 5235** для одного из процессов был изменён приоритет.

Его значение изменилось с **0** на **5** (понижение приоритета, процесс будет получать меньше ресурсов CPU).

- Повторная команда **renice -n 15 5235** изменила приоритет того же процесса с **5** на **15**.

Теперь процесс ещё ниже в очереди планировщика, что означает ещё меньшее выделение процессорного времени.

Разница между значениями **-5** и **-15** в том, что чем меньше число `nice` (включая отрицательные значения), тем выше приоритет.

В данном случае использование положительных значений увеличило «доброту» процесса, уступая ресурсы другим задачам.

4. Для завершения всех запущенных процессов **dd** применена команда **killall dd**.

Все три процесса были успешно остановлены.

```
rkogengar@rishardkogengar:~$  
rkogengar@rishardkogengar:~$ su  
Password:  
root@rishardkogengar:/home/rkogengar# dd if=/dev/zero of=/dev/null &  
[1] 5235  
root@rishardkogengar:/home/rkogengar# dd if=/dev/zero of=/dev/null &  
[2] 5247  
root@rishardkogengar:/home/rkogengar# dd if=/dev/zero of=/dev/null &  
[3] 5249  
root@rishardkogengar:/home/rkogengar# renice -n 5 5235  
5235 (process ID) old priority 0, new priority 5  
root@rishardkogengar:/home/rkogengar# renice -n 15 5235  
5235 (process ID) old priority 5, new priority 15  
root@rishardkogengar:/home/rkogengar# killall dd  
[1] Terminated dd if=/dev/zero of=/dev/null  
[2]- Terminated dd if=/dev/zero of=/dev/null  
[3]+ Terminated dd if=/dev/zero of=/dev/null  
root@rishardkogengar:/home/rkogengar#
```

Рис. 2.6: Работа с приоритетами dd

2.4 Задание 2. Управление процессами с программой

yes

1. Запущена программа **yes** в фоновом режиме с подавлением вывода в `/dev/null`.

Программа работает в фоне и не мешает использованию терминала.

2. Программа **yes** была запущена на переднем плане с перенаправлением вывода.

Её выполнение приостановлено с помощью **Ctrl+Z**, затем она была возвращена в работу через **fg**, после чего завершена комбинацией **Ctrl+C**.

3. Вновь запущена программа **yes**, но уже без перенаправления вывода.

После приостановки процесса его можно было возобновить и завершить аналогичным образом.

```
root@rishardkogengar:/home/rkogengar#
root@rishardkogengar:/home/rkogengar# yes > /dev/null &
[1] 5462
root@rishardkogengar:/home/rkogengar# yes > /dev/null
^Z
[2]+  Stopped                  yes > /dev/null
root@rishardkogengar:/home/rkogengar# fg 2
yes > /dev/null
^C
root@rishardkogengar:/home/rkogengar# █
```

Рис. 2.7: yes без перенаправления

4. Проверка состояния заданий командой **jobs** показала список процессов с их статусами (**Running** или **Stopped**).
5. Один из фоновых процессов был переведён на передний план командой **fg** и завершён.

Другой процесс наоборот, отправлен в фоновый режим через **bg**.

```
root@rishardkogengar:/home/rkogengar#
root@rishardkogengar:/home/rkogengar# jobs
[1]+  Running                  yes > /dev/null &
root@rishardkogengar:/home/rkogengar# fg 1
yes > /dev/null
^Z
[1]+  Stopped                  yes > /dev/null
root@rishardkogengar:/home/rkogengar# bg 1
[1]+ yes > /dev/null &
root@rishardkogengar:/home/rkogengar# jobs
[1]+  Running                  yes > /dev/null &
root@rishardkogengar:/home/rkogengar# nohup yes > /dev/null &
[2] 6121
nohup: ignoring input and redirecting stderr to stdout
root@rishardkogengar:/home/rkogengar# █
```

Рис. 2.8: Управление заданиями

6. Запущен процесс **yes** с использованием **nohup**, что позволяет ему продолжить работу даже после выхода из терминала.

Подтверждение работы процесса было получено с помощью его PID.

7. В новой сессии консоли утилита **top** показала активные процессы.

Видно, что несколько процессов **yes** активно загружают CPU.

```
rkogengar@rishardkogengar:/home/rkogengar - top
top - 18:20:10 up 24 min,  5 users,  load average: 1.87, 1.25, 0.72
Tasks: 266 total,  4 running, 262 sleeping,  0 stopped,  0 zombie
%Cpu(s): 19.0 us, 33.3 sy,  0.0 ni, 47.6 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem : 3909.0 total, 1398.0 free, 1319.0 used, 1431.8 buff/cache
MiB Swap: 4040.0 total, 4040.0 free,  0.0 used, 2590.0 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR S %CPU  %MEM    TIME+  COMMAND
 5462 root        20   0 226820 1748 1748 R 90.9   0.0   6:50.10 yes
 6121 root        20   0 226820 1732 1732 R 81.8   0.0   1:48.54 yes
    1 root         0 -20  49192 41044 10248 S  0.0   1.0   0:02.46 systemd
    2 root         0  0         0     0     0 S  0.0   0.0   0:00.00 kthread
    3 root         0  0         0     0     0 S  0.0   0.0   0:00.00 pool_workqueue_release
    4 root         0 -20         0     0     0 I  0.0   0.0   0:00.00 kworker/R-rcu_gp
    5 root         0 -20         0     0     0 I  0.0   0.0   0:00.00 kworker/R-sync_wq
    6 root         0 -20         0     0     0 I  0.0   0.0   0:00.00 kworker/R-slab_flushwq
    7 root         0 -20         0     0     0 I  0.0   0.0   0:00.00 kworker/R-netns
    9 root        20   0         0     0     0 I  0.0   0.0   0:00.08 kworker/0:1-cgroup_destroy
   10 root         0 -20         0     0     0 I  0.0   0.0   0:00.00 kworker/0:0H-events_highp+
   11 root        20   0         0     0     0 I  0.0   0.0   0:00.00 kworker/u16:0-events_unbo+
   12 root        20   0         0     0     0 I  0.0   0.0   0:00.10 kworker/u16:1-netns
   13 root         0 -20         0     0     0 I  0.0   0.0   0:00.00 kworker/R-mm_percpu_wq
   14 root        20   0         0     0     0 I  0.0   0.0   0:00.00 rcu_tasks_kthread
   15 root        20   0         0     0     0 I  0.0   0.0   0:00.00 rcu_tasks_rude_kthread
   16 root        20   0         0     0     0 I  0.0   0.0   0:00.00 rcu_tasks_trace_kthread
   17 root        20   0         0     0     0 S  0.0   0.0   0:00.01 ksoftirqd/0
   18 root        20   0         0     0     0 I  0.0   0.0   0:00.17 rcu_preempt
   19 root        20   0         0     0     0 S  0.0   0.0   0:00.00 rcu_exp_par_gp_kthread_wo+
   20 root        20   0         0     0     0 S  0.0   0.0   0:00.12 rcu_exp_gp_kthread_worker
   21 root         rt   0         0     0     0 S  0.0   0.0   0:00.01 migration/0
   22 root       -51   0         0     0     0 S  0.0   0.0   0:00.00 idle_inject/0
   23 root        20   0         0     0     0 S  0.0   0.0   0:00.00 cpuhp/0
   24 root        20   0         0     0     0 S  0.0   0.0   0:00.00 cpuhp/1
   25 root       -51   0         0     0     0 S  0.0   0.0   0:00.00 idle_inject/1
   26 root         rt   0         0     0     0 S  0.0   0.0   0:00.22 migration/1
   27 root        20   0         0     0     0 S  0.0   0.0   0:00.01 ksoftirqd/1
   29 root         0 -20         0     0     0 I  0.0   0.0   0:00.00 kworker/1:0H-events_highp+
```

Рис. 2.9: Мониторинг процессов top

8. Запущено три дополнительных процесса **yes** в фоновом режиме.

Два из них были завершены разными способами:

- по PID с помощью **kill**;
- по идентификатору задания с помощью **fg** и **Ctrl+C**.

```

root@rishardkogengar:/home/rkogengar# yes > /dev/null &
[1] 6582
root@rishardkogengar:/home/rkogengar# yes > /dev/null &
[2] 6584
root@rishardkogengar:/home/rkogengar# yes > /dev/null &
[3] 6587
root@rishardkogengar:/home/rkogengar# kill 6582
[1] Terminated          yes > /dev/null
root@rishardkogengar:/home/rkogengar# fg 2
yes > /dev/null
^C
root@rishardkogengar:/home/rkogengar# kill -1 5462
root@rishardkogengar:/home/rkogengar# kill -1 6582
bash: kill: (6582) - No such process
root@rishardkogengar:/home/rkogengar# yes > /dev/null &
[4] 6797
root@rishardkogengar:/home/rkogengar# yes > /dev/null &
[5] 6799
root@rishardkogengar:/home/rkogengar# yes > /dev/null &
[6] 6811
root@rishardkogengar:/home/rkogengar# killall yes
[4] Terminated          yes > /dev/null
[3] Terminated          yes > /dev/null
[5]- Terminated          yes > /dev/null
[6]+ Terminated          yes > /dev/null
root@rishardkogengar:/home/rkogengar# █

```

Рис. 2.10: Завершение процессов

9. Проверена работа сигналов. Обычный процесс был завершён при получении сигнала **SIGHUP**, тогда как процесс, запущенный через **nohup**, продолжил работу.
10. Дополнительно запущено несколько процессов **yes**, которые были завершены одновременно командой **killall yes**.
11. Запущены два процесса **yes**: один обычный, другой с изменённым приоритетом с помощью **nice -n 5**.
С помощью команды **ps -l** сравнивались их приоритеты (поле NI).
Видно, что у процесса, запущенного с **nice -n 5**, приоритет выше (он более «добрый» и уступает ресурсы).
12. Один из процессов получил новое значение приоритета с помощью команды **renice**.
В результате приоритеты обоих процессов сравнялись.

```

root@rishardkogengar:/home/rkogengar# yes > /dev/null &
[1] 6872
root@rishardkogengar:/home/rkogengar# nice -n 5 yes > /dev/null &
[2] 6895
root@rishardkogengar:/home/rkogengar# ps -l
F S  UID      PID     PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S   0        6419    6384  0  80   0 - 58153 do_wai pts/1      00:00:00 su
4 S   0        6430    6419  0  80   0 - 57575 do_wai pts/1      00:00:00 bash
4 R   0        6872    6430 99  80   0 - 56705 -      pts/1      00:00:11 yes
4 R   0        6895    6430 98  85   5 - 56705 -      pts/1      00:00:04 yes
4 R   0        6897    6430  0  80   0 - 57682 -      pts/1      00:00:00 ps
root@rishardkogengar:/home/rkogengar# renice -n 5 6872
6872 (process ID) old priority 0, new priority 5
root@rishardkogengar:/home/rkogengar# ps -l
F S  UID      PID     PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S   0        6419    6384  0  80   0 - 58153 do_wai pts/1      00:00:00 su
4 S   0        6430    6419  0  80   0 - 57575 do_wai pts/1      00:00:00 bash
4 R   0        6872    6430 99  85   5 - 56705 -      pts/1      00:00:42 yes
4 R   0        6895    6430 99  85   5 - 56705 -      pts/1      00:00:34 yes
4 R   0        6965    6430  0  80   0 - 57682 -      pts/1      00:00:00 ps
root@rishardkogengar:/home/rkogengar# killall yes
[1]-  Terminated                  yes > /dev/null
[2]+  Terminated                  nice -n 5 yes > /dev/null
root@rishardkogengar:/home/rkogengar# █

```

Рис. 2.11: Выравнивание приоритетов

3 Контрольные вопросы

1. Обзор всех текущих заданий оболочки показывает команда **jobs**.
2. Чтобы остановить текущее задание и продолжить его выполнение в фоновом режиме, используют сочетание клавиш **Ctrl+Z**, а затем команду **bg**.
3. Для отмены текущего задания применяется комбинация клавиш **Ctrl+C**.
4. Если доступ к оболочке недоступен, задание можно отменить, используя другую консоль и команду **kill** с указанием PID процесса или **killall** с именем команды.
5. Для отображения отношений между родительскими и дочерними процессами используется команда **ps fax**.
6. Изменить приоритет процесса с идентификатором 1234 на более высокий можно командой **renice -n -5 -p 1234**.
7. Если запущено много процессов dd (например, 20), проще всего завершить их командой **killall dd**.
8. Для остановки команды с именем *mycommand* используется команда **killall mycommand**.
9. В утилите **top** для завершения процесса применяется клавиша **k**, после чего вводится PID нужного процесса.

10. Чтобы запустить команду с достаточно высоким приоритетом, но при этом не перегрузить систему, используется команда **nice** с положительным значением (например, **nice -n 10 команда**). Это позволяет процессу работать, но отдавать ресурсы другим задачам при необходимости.

4 Заключение

В ходе лабораторной работы были освоены основные приёмы управления процессами в Linux.